5.1 Основы конкурентности в Go

Конкурентность — это способность системы управлять несколькими задачами одновременно, даже если они не выполняются параллельно

КОНКУРЕНТНОСТЬ != ПАРАЛЛЕЛИЗМ

В кофейне один бариста, он отвечает за прием заказов и их приготовление с помощью одной кофемашины. Клиенты делают заказы, а затем ждут кофе:



Очередь из клиентов

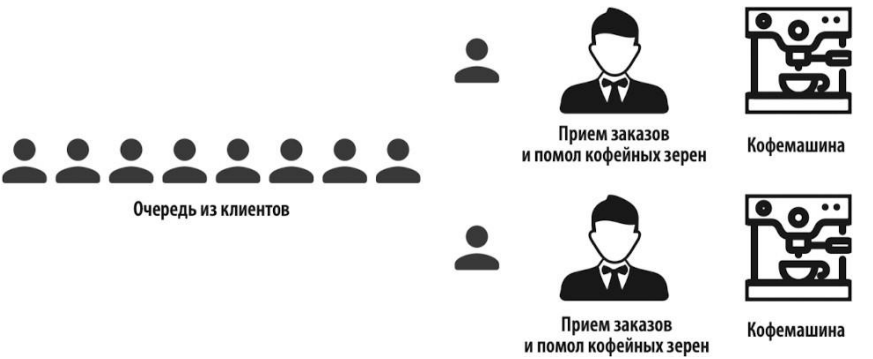


Прием заказов
и помол кофейных зерен



Кофемашина

Можно нанять второго бариста для ускорения общего процесса. Параллельная реализация системы обслуживания:



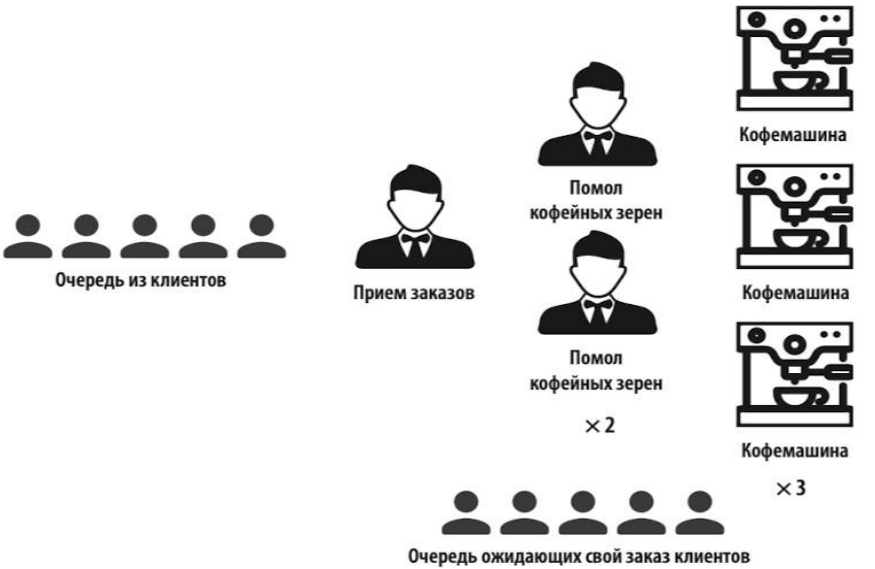
Чтобы не заставлять клиентов ждать в очереди можно ввести еще одну очередь — для клиентов, ожидающих выполнения принятых заказов:



Добавим элемент параллелизма для шага помолки зерен:



Добавим дополнительные кофемашины:



```
Конкурентность — это о работе с большим количеством
вещей одновременно.

Параллелизм — это о выполнении множества дел
одновременно.
```

Роб Пайк

Зачем Google создали Go?

Go - решение проблем масштабируемости и производительности в серверных системах (Go History):

- **Простота**: Синтаксис, похожий на C, но без лишней сложности.
- **Скорость**: Компиляция в машинный код для высокой производительности.
- **Масштабируемость**: Подходит для систем с тысячами соединений.

CPU-bound и IO-bound нагрузки

- **CPU-bound**: Задачи, интенсивно использующие процессор (например, вычисления, рендеринг).
- **IO-bound**: Задачи, зависящие от ввода-вывода (например, сетевые запросы, чтение файлов).

Go эффективен для **IO-bound задач**. Планировщик позволяет обрабатывать множество операций без создания тяжелых потоков

(Concurrency Patterns).

Потоки ОС: Что это и почему они дорогие?

Поток — это наименьшая единица обработки, которую может выполнять операционная система (ОС). Если процесс хочет выполнять несколько действий одновременно, он запускает несколько потоков.

Почему потоки ОС дорогие?

Потоки операционной системы имеют высокие накладные расходы:

- **Память**: Стек потока занимает 1-8 МБ.
- **Управление**: Создание и завершение потоков требуют значительных ресурсов.
- **Контекстные переключения**: Переключение между потоками замедляет выполнение из-за сохранения состояния.

ОС отвечает за планирование потоков:

Все потоки могли использовать процессорное время, не ожидая слишком долго.

Рабочая нагрузка равномерно распределялась между ядрами процессора

Переключение контекста остается дорогостоящей операцией (Go Scheduler).

Почему контекстное переключение медленное?

Контекстное переключение между потоками ОС замедляет работу, потому что:

- **Сохранение состояния**: ОС сохраняет текущее состояние выполнения потока (например, значения регистров).
- **Кэш-память**: Переключение инвалидирует кэши L1/L2, вызывая промахи и задержки.
- **Планирование**: ОС тратит время на выбор следующего потока для выполнения.

Эти накладные расходы делают традиционные потоки менее эффективными для высококонкурентных приложений.

User space и kernel space

- **Kernel space**: Область ядра ОС, где выполняются системные вызовы и управление оборудованием. Переключение в kernel space дорого.
- **User space**: Область для приложений, без прямого доступа к ядру. Go управляет горутинами в **user space**, что снижает количество системных вызовов и повышает эффективность

(Go Runtime). Переключение горутин на 80–90% быстрее, чем переключение потоков ОС, так как оно происходит без участия ядра.

Ключевые идеи реализации в Go

Go абстрагирует сложность потоков ОС:

- **Горутины вместо потоков**: Программист работает с горутинами, не думая о потоках.
- **Легкость**: Горутины дешевле (стек ~2 КБ) и быстрее создаются, чем потоки ОС.
- **Кооперативность**: Горутины уступают управление в определенных точках (например, при ожидании I/O), упрощая синхронизацию.

Горутины — это легковесные потоки, управляемые runtime Go, а не ОС.

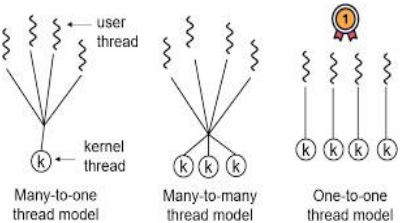
Жизненный цикл горутины

Горутина имеет более простой жизненный цикл, чем поток ОС:

- **Исполнение (executing)**: Горутина выполняется на потоке ОС.
- **Готовность к выполнению (runnable)**: Горутина ожидает назначения на поток для выполнения.
- **Ожидание (waiting)**: Горутина остановлена, ожидая завершения операции (например, системного вызова или мьютекса).

Эти состояния помогают Go эффективно управлять горутинами.

N:1 и N:M модели



- **N:1**: Много горутин на один поток ОС. Простая модель, но не использует многоядерность, так как все горутины выполняются в одном потоке.
- **N:M**: Много горутин на много потоков ОС. Go использует эту модель для параллелизма.

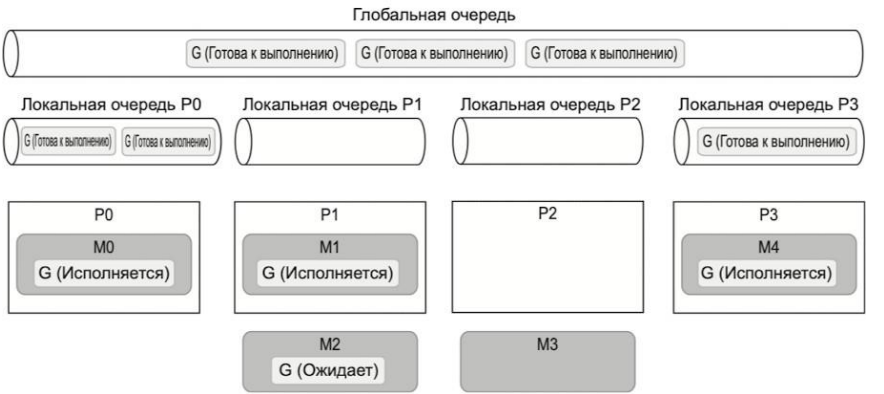
Runtime Go распределяет горутины (N) по потокам ОС (M) через планировщик, что позволяет эффективно использовать многоядерные системы (GMP Model).

Концепции рантайма Go: Модель GMP

Модель **GMP** (Goroutine, Machine, Processor) — основа конкурентности в Go:

- **G (Goroutine)**: Легковесный поток, единица выполнения.
- **M (Machine)**: Поток ОС, выполняющий горутины.
- **P (Processor)**: Логический процессор, управляющий локальной очередью горутин.

Как работает GMP:



- Каждый **P** имеет локальную очередь горутин и привязан к одному **M**. Горутины распределяются между **глобальной очередью** (для всех P) и **локальными очередями** (для каждого P).
- Если локальная очередь пуста, P может "украсть" горутины из других очередей (work stealing).
- Количество P определяется настройкой `GOMAXPROCS` , по умолчанию равной числу ядер CPU.

Что происходит, когда горутина не может быть выполнена?

Если все M заняты, новая горутина ставится в очередь. Go поддерживает:

- **Локальные очереди**: По одной для каждого P.
- **Глобальная очередь**: Общая для всех P.

Компонент	Описание	Роль
G	Горутина	Легковесная единица выполнения, стек ~2 КБ
M	Поток ОС	Выполняет горутины, управляется ОС
P	Логический процессор	Управляет локальной очередью, распределяет горутины

Планировщик в Go

- **Кооперативный**: Горутины уступают управление в точках, таких как вызовы функций или ожидание I/O.
- **Преемпция**: С Go 1.14 планировщик может прерывать горутины, выполняющиеся дольше 10 мс, чтобы избежать starvation (Go 1.14 Notes).
- **Эффективность**: Переключение горутин (200 нс) быстрее, чем потоков ОС (1000 нс).

Планировщик Go обеспечивает баланс между горутинами, минимизируя простои CPU.

Что такое work stealing?

Work stealing (иногда называемый work sharing) — это механизм балансировки нагрузки:

- Если P простаивает, он "крадет" горутины из локальных очередей других P или глобальной очереди.
- Пример: Если P0 занят долгой горутиной, P1 может взять задачи из очереди P0.

Почему это важно?

- Обеспечивает равномерное использование всех ядер CPU.
- Снижает задержки, связанные с ожиданием завершения задач.

Work stealing делает систему более эффективной, особенно для IO-bound задач.

GOMAXPROCS и его роль

`GOMAXPROCS` определяет количество потоков ОС, которые могут выполнять код Go одновременно (GOMAXPROCS Docs):

- **По умолчанию**: Равно числу ядер CPU.
- **Настройка**: Увеличение может помочь для IO-bound задач, уменьшение — для CPU-bound.
- **Влияние**: Контролирует количество P, влияя на параллелизм.

Практические аспекты

- **IO-bound задачи**: Горутины идеальны для обработки сетевых запросов или ожидания данных.
- **Настройка GOMAXPROCS**: Помогает оптимизировать производительность на многоядерных системах.
- **Избегание проблем**: Знание преемпции и work stealing помогает предотвратить блокировки и starvation.

Заключение

Go предлагает мощную и простую модель конкурентности через горутины, каналы и модель GMP. Она абстрагирует сложность потоков ОС, оптимизирует IO-bound задачи и масштабируется на многоядерных системах. Понимание этих концепций помогает писать эффективные программы.

Ссылки

- Go FAQ: History of Go
- Go Concurrency Patterns Talk Ardan Labs: Scheduling in Go Part II Go Runtime Source
- SoByte: GMP Model for Go Scheduler Go 1.14 Release Notes
- Bytesize Go: Go Scheduler Deep Dive 2025
- Go GOMAXPROCS Documentation